



MELIKA

Plataforma Web de Gestión de Citas Médicas

DOCUMENTACIÓN TÉCNICA

MÓDULO FRONTEND · React + Vite

Campo	Detalle
Proyecto	MELIKA — Sistema de Agendamiento y Gestión Clínica
Repositorio	github.com/Imjuaniss/proyecto-melika
Módulo documentado	Frontend (carpeta /client)
Tecnología principal	React 19 · Vite 8 · JavaScript ES2023 · CSS puro
Entorno de despliegue	Railway (PaaS) — servicio independiente del backend
Versión del documento	1.0
Fecha	Julio de 2026

Historial de Revisión

Fecha	Versión	Descripción	Autor
Julio 2026	1.0	Elaboración inicial de la documentación técnica del frontend de MELIKA: arquitectura, stack, estructura de archivos, autenticación, enrutamiento, componentes, validaciones y despliegue.	Equipo de Desarrollo MELIKA

Índice de Contenidos

Índice de Contenidos	3
1. Descripción General del Frontend.....	5
1.1 Objetivos del Módulo Frontend	5
1.2 Stack Tecnológico	5
1.3 Arquitectura y Principios de Diseño.....	6
Principios aplicados	7
Diagrama conceptual de capas.....	7
1.4 Estructura de Carpetas y Archivos	8
1.5 Sistema de Diseño (Design Tokens)	11
Paleta de color	11
Tipografía	11
Escala de espaciado	11
1.6 Gestión de Estado y Autenticación en el Cliente	11
Flujo de autenticación	12
1.7 Enrutamiento y Control de Acceso por Rol	12
1.8 Comunicación con el Backend (Cliente API)	14
1.9 Módulos Funcionales Principales	14
Autenticación y gestión de cuenta	14
Agendamiento de citas (paciente)	15
Historias clínicas y cumplimiento normativo.....	15
Panel del médico.....	15
Gestión de horarios	15
Panel administrativo.....	15
Catálogo público de medicamentos y especialidades	15
1.10 Componentes de Interfaz Reutilizables	15
1.11 Generación y Visualización de Documentos PDF.....	17
1.12 Validación de Datos en el Cliente.....	17
Reglas implementadas.....	17
1.13 Configuración del Entorno de Build (Vite)	18
1.14 Variables de Entorno	18
1.15 Despliegue en Railway.....	19
Configuración de despliegue (client/railway.json)	19
Características provistas por la plataforma	19
1.16 Calidad de Código y Convenciones.....	19

1.17 Consideraciones y Oportunidades de Mejora	20
---	-----------

1. Descripción General del Frontend

El frontend de MELIKA constituye la capa de presentación e interacción de una plataforma web orientada a la gestión integral de citas médicas, historias clínicas y administración de un centro de salud. Está construido como una Single Page Application (SPA) sobre React 19, utilizando Vite como motor de desarrollo y empaquetado, y se comunica de forma exclusiva con el backend a través de una API REST propia, consumida mediante la Fetch API nativa del navegador.

A diferencia de una aplicación de un solo perfil de usuario, MELIKA implementa una interfaz adaptativa por rol: la misma base de código sirve tres experiencias diferenciadas —paciente, médico y administrador— controlando de forma dinámica la navegación visible, las rutas accesibles y las acciones disponibles según el rol autenticado. Esta decisión de arquitectura reduce la duplicación de código y centraliza la lógica de negocio del lado del cliente en un único conjunto de componentes reutilizables.

El proyecto no depende de frameworks de estilos (Bootstrap, Tailwind, Material UI, etc.): toda la interfaz se construye sobre CSS puro, apoyada en un sistema de diseño propio basado en variables CSS (design tokens), lo que garantiza consistencia visual, tiempos de carga reducidos y control total sobre la identidad de marca de MELIKA.

Enfoque del documento

Este documento describe exclusivamente el módulo Frontend del proyecto MELIKA (carpeta /client del repositorio). El backend (Node.js, Express 5, PostgreSQL) se referencia únicamente en los puntos donde el frontend depende directamente de sus contratos de API.

1.1 Objetivos del Módulo Frontend

- Ofrecer una interfaz clara, accesible y responsiva para que los pacientes agenden citas médicas, consulten su historial y gestionen su cuenta.
- Proveer a los médicos un panel de trabajo para consultar su agenda, gestionar horarios y registrar historias clínicas de forma normativa.
- Brindar al equipo administrativo un panel de control completo (CRUD) sobre médicos, especialidades, horarios, medicamentos, usuarios y citas.
- Garantizar la seguridad de la sesión mediante autenticación basada en JSON Web Tokens (JWT), con control de acceso a rutas según el rol del usuario.
- Mantener una arquitectura desacoplada del backend, comunicándose únicamente a través de contratos HTTP/JSON bien definidos.

1.2 Stack Tecnológico

La siguiente tabla resume las tecnologías, librerías y herramientas que conforman el ecosistema del frontend de MELIKA, según se encuentran declaradas en client/package.json y en la configuración del proyecto:

Categoría	Tecnología	Rol dentro del proyecto
Librería de interfaz	React 19.2	Construcción de la interfaz mediante componentes funcionales y Hooks (useState, useEffect, useContext, useCallback).
Entorno de build / dev server	Vite 8	Servidor de desarrollo con Hot Module Replacement (HMR) y empaquetado optimizado para producción (build a carpeta dist/).
Enrutamiento SPA	React Router DOM 7	Enrutamiento declarativo del lado del cliente, con rutas anidadas (panel /admin) y componentes de guardia por rol.
Lenguaje	JavaScript ES2023 (Vanilla JS)	Sin TypeScript; módulos ES nativos (import/export), sintaxis async/await para las llamadas a la API.
Estilos	CSS3 puro	Sin frameworks de UI. Hoja de variables global (styles/index.css) + hojas de estilo dedicadas por componente/página.
Gestión de estado global	Context API de React	AuthContext centraliza la sesión del usuario (token, datos, rol) sin librerías externas de estado (Redux, Zustand, etc.).
Comunicación con la API	Fetch API nativa	Cliente HTTP propio (lib/apiClient.js) que añade cabeceras JSON y el token JWT automáticamente.
Calendarios y agenda	FullCalendar 6 (core, daygrid, timegrid, interaction, react)	Visualización interactiva de horarios y franjas disponibles para el agendamiento de citas.
Generación de documentos	@react-pdf/renderer 4	Renderizado vectorial de historias clínicas en formato PDF, generado 100% en el cliente.
Visualización de PDF	@pdfslick/react 4	Visor de documentos PDF embebido en el navegador (VisorPDFModal).
Carruseles / sliders	Swiper 14	Componente de carrusel para banners y contenido destacado en la página de Inicio.
Calidad de código	ESLint 10 + eslint-plugin-react-hooks + eslint-plugin-react-refresh	Verificación estática de buenas prácticas en el uso de Hooks y compatibilidad con Fast Refresh.
Tipografía	Google Fonts — Sora, Lato, Source Code Pro	Fuente Sora para títulos y elementos de marca, Lato para texto de cuerpo, Source Code Pro para contenido monoespaciado.
Iconografía	Sprite SVG propio (public/icons.svg)	Conjunto de iconos vectoriales propios, sin dependencia de librerías externas de iconos.
Servidor de archivos estáticos (producción)	serve 14	Sirve el resultado de vite build en producción mediante npm start.
Despliegue	Railway (PaaS, builder Nixpacks)	Servicio independiente del backend, con build y despliegue continuo desde el repositorio.

1.3 Arquitectura y Principios de Diseño

El frontend sigue una arquitectura por capas y por características (feature-based), separando con claridad tres responsabilidades: presentación de vistas (pages), componentes reutilizables de interfaz (components) y acceso a datos (lib y services). Esta organización permite que cada página de la aplicación sea autocontenida —con su propio archivo JSX y su propia hoja de estilos CSS colocada junto a él— facilitando el mantenimiento y la incorporación de nuevos módulos sin afectar el resto del sistema.

Principios aplicados

- **Component-based UI:** toda la interfaz se descompone en componentes funcionales de React, favoreciendo la reutilización (por ejemplo, Boton, DatePicker y TimePicker se emplean en múltiples formularios del sistema).
- **Colocalización de estilos:** cada componente o página incluye su propio archivo .css con el mismo nombre (por ejemplo, Login.jsx junto a Login.css), evitando hojas de estilo monolíticas difíciles de mantener.
- **Renderizado condicional por rol:** componentes como la barra de navegación (Navbar) y las rutas de la aplicación (App.jsx) adaptan su contenido según el rol del usuario autenticado (paciente, médico o administrador), en lugar de mantener aplicaciones o vistas separadas por perfil.
- **Single Source of Truth de sesión:** el estado de autenticación vive en un único proveedor de contexto (AuthContext) consultado por toda la aplicación mediante el hook useAuth.
- **Cliente HTTP centralizado:** las peticiones a la API pasan por un único módulo (apiClient.js) que estandariza cabeceras, manejo de errores y adjunción del token JWT.
- **Validación espejo (Frontend/Backend):** las reglas de validación clínica se replican intencionalmente en el cliente (utils/validacionClinica.js) y en el servidor, para ofrecer retroalimentación inmediata al usuario sin sacrificar la validación de seguridad del lado del servidor.

Diagrama conceptual de capas

Capa	Carpeta	Responsabilidad
Enrutamiento y control de acceso	App.jsx	Define las rutas de la SPA y aplica guardas (RutaPaciente, RutaMedico, RutaAdmin) según el rol.
Vistas / Páginas	src/pages/	Una carpeta por vista funcional; combina componentes de UI con llamadas a la API.
Componentes reutilizables	src/components/	Elementos de interfaz usados en más de una vista: navegación, formularios clínicos, selectores de fecha/hora, calendarios.
Estado global	src/context/	AuthContext: sesión, token y datos del usuario autenticado.
Acceso a datos	src/lib/ · src/services/	apiClient.js (cliente HTTP genérico) y servicios específicos por dominio (especialidadesService.js).
Utilidades	src/utils/	Funciones puras de validación y soporte, independientes de la interfaz.

Capa	Carpeta	Responsabilidad
Diseño visual	src/styles/	Variables CSS globales (colores, tipografía, espaciado) que alimentan todas las hojas de estilo del proyecto.

1.4 Estructura de Carpetas y Archivos

El frontend se organiza dentro de la carpeta /client del repositorio, siguiendo la estructura estándar de un proyecto Vite + React, extendida con una organización por dominio funcional dentro de src/pages y src/components. La siguiente tabla detalla los archivos y carpetas más relevantes del proyecto:

Archivo / Carpeta	Descripción
App.jsx	Componente raíz: define el árbol de rutas de la SPA y los componentes de guardia de acceso por rol (RutaPaciente, RutaMedico, RutaAdmin).
main.jsx	Punto de entrada de la aplicación: monta el componente App sobre el DOM mediante createRoot.
context/AuthContext.jsx	Proveedor de contexto de autenticación: expone usuario, login() y logout(), y persiste la sesión en localStorage.
lib/apiClient.js	Cliente HTTP centralizado sobre Fetch API: añade cabeceras JSON y token Bearer, estandariza el manejo de errores del backend.
services/especialidadesService.js	Servicio de acceso a datos para el dominio de especialidades médicas y médicos asociados.
utils/validacionClinica.js	Validaciones clínicas en tiempo real (formato CIE-10, detección de texto no descriptivo, validación de nombres propios), sincronizadas con las reglas del backend.
styles/index.css	Hoja de estilos raíz: variables CSS (design tokens) de color, tipografía y espaciado utilizadas por todo el proyecto.
components/layout/Navbar.jsx	Barra de navegación principal con renderizado condicional de enlaces y acciones según el rol del usuario.
components/ui/Boton.jsx	Componente de botón reutilizable con variantes (primary, ghost, outline) y tamaños configurables.
components/ui/DatePicker.jsx	Selector de fecha personalizado utilizado en formularios de agendamiento y horarios.
components/ui/TimePicker.jsx	Selector de hora personalizado para la configuración de franjas de atención.
components/ui/ClockPicker.jsx	Selector de hora en formato de reloj interactivo, empleado en la configuración semanal de horarios.

Archivo / Carpeta	Descripción
components/horarios/HorariosSemana.jsx	Configurador semanal de horarios de atención médica, con selección de días y franjas mediante ClockPicker.
components/historias/HistorialPaciente.jsx	Listado del historial de historias clínicas asociadas a un paciente.
components/historias/ModalHistoriaClinica.jsx	Formulario modal para el registro y edición de una historia clínica.
components/historias/FormularioAclaracion.jsx	Formulario global de aclaración clínica, montado una única vez en App.jsx y activado mediante eventos personalizados del navegador.
components/historias/PlantillaHistoriaPDF.jsx	Plantilla vectorial de la historia clínica en formato PDF, construida con @react-pdf/renderer conforme a la normativa colombiana vigente.
components/historias/VisorPDFModal.jsx	Visor de documentos PDF embebido, basado en @pdfslick/react.
components/SliderInicio.jsx	Carrusel de contenido destacado en la página de inicio, implementado con Swiper.
pages/inicio/Inicio.jsx	Página pública de bienvenida: presentación del centro médico, especialidades destacadas y carrusel informativo.
pages/login/Login.jsx	Formulario de inicio de sesión; redirige automáticamente según el rol devuelto por la API.
pages/registro/Registro.jsx	Formulario de registro de nuevos pacientes.
pages/verificar/Verificar.jsx	Verificación de cuenta mediante código enviado por correo electrónico.
pages/activar-cuenta/ActivarCuenta.jsx	Flujo de activación de cuenta a partir de un enlace o token recibido por correo.
pages/recuperar/RecuperarPassword.jsx	Solicitud de recuperación de contraseña olvidada.
pages/recuperar/NuevaPassword.jsx	Formulario de definición de una nueva contraseña tras la recuperación.
pages/dashboard/Dashboard.jsx	Panel principal del paciente: accesos rápidos a agendamiento, citas y perfil.
pages/agendar/Agendarcita.jsx	Flujo de agendamiento de citas: selección de especialidad, médico, fecha y franja horaria disponible.
pages/miscitas/MisCitas.jsx	Listado e historial de citas del paciente autenticado, con acceso a sus historias clínicas.
pages/dashboard-medico/DashboardMedico.jsx	Panel del médico: agenda del día, gestión de citas asignadas y acceso al registro de historias clínicas.

Archivo / Carpeta	Descripción
pages/catalogo/Catalogo.jsx	Catálogo público de medicamentos ofrecidos por la institución.
pages/especialidades/Especialidades.jsx	Listado de especialidades médicas disponibles.
pages/especialidades/MedicosEspecialidad.jsx	Listado de médicos asociados a una especialidad seleccionada.
pages/admin/AdminLayout.jsx	Layout contenedor del panel administrativo; define la navegación lateral y renderiza las subrutinas de gestión.
pages/admin/dashboard/AdminDashboard.jsx	Panel de métricas generales del sistema para el rol administrador.
pages/admin/gestionmedicos/MedicosAdmin.jsx	CRUD de médicos: creación, edición, activación/inactivación y consulta.
pages/admin/gestionhorarios/HorariosAdmin.jsx	Gestión administrativa de horarios de atención por médico.
pages/admin/gestionusuarios/UsuariosAdmin.jsx	Administración de usuarios registrados en la plataforma.
pages/admin/gestioncitas/CitasAdmin.jsx	Supervisión y gestión de todas las citas agendadas en el sistema.
pages/admin/gestionespecialidades/EspecialidadesAdmin.jsx	CRUD de especialidades médicas.
pages/admin/gestionmedicamentos/MedicamentosAdmin.jsx	CRUD del catálogo de medicamentos.
public/icons.svg	Sprite SVG con el conjunto de iconos propios de la interfaz.
vite.config.js	Configuración de Vite: alias de módulos, formato del worker para PDF.js y optimización de dependencias.
railway.json	Configuración de despliegue en Railway: builder Nixpacks, comando de build y comando de arranque.
eslint.config.js	Reglas de linting del proyecto (React Hooks, React Refresh).

1.5 Sistema de Diseño (Design Tokens)

MELIKA cuenta con un sistema de diseño propio, definido íntegramente en variables CSS (custom properties) dentro de `src/styles/index.css`. Este enfoque —equivalente a los "design tokens" de sistemas de diseño profesionales como Material Design o Carbon— permite mantener consistencia visual en toda la aplicación y facilita futuros cambios de marca modificando un único archivo.

Paleta de color

Grupo	Uso	Ejemplo de variable
Azules primarios	Color corporativo de MELIKA: fondos oscuros, textos principales, botones primarios y encabezados.	<code>--melika-primary-900</code> → <code>--melika-primary-25</code>
Coral / naranja (acento)	Color de acción: botones de llamado a la acción, énfasis y elementos interactivos destacados.	<code>--melika-accent</code> , <code>--melika-accent-dark</code>
Fondos	Fondos de página y de tarjetas.	<code>--melika-bg</code> , <code>--melika-bg-soft</code> , <code>--melika-bg-secondary</code>
Semánticos	Estados de éxito, advertencia y error en formularios y notificaciones.	<code>--melika-success</code> , <code>--melika-warning</code> , <code>--melika-error</code>
Texto	Jerarquía tipográfica de texto sobre fondo claro.	<code>--melika-text-primary</code> , <code>--melika-text-secondary</code> , <code>--melika-text-muted</code>

Tipografía

El proyecto utiliza tres familias tipográficas de Google Fonts, cada una con un propósito específico dentro de la jerarquía visual:

- **Sora (`--font-display`):** utilizada en títulos, marca y elementos destacados de la interfaz.
- **Lato (`--font-body`):** tipografía de lectura para párrafos, formularios y contenido general.
- **Source Code Pro (`--font-mono`):** reservada para contenido de tipo técnico o códigos (por ejemplo, códigos de verificación).

Escala de espaciado

El sistema define una escala de espaciado basada en múltiplos de 4px (`--space-1` a `--space-24`), lo que asegura consistencia en márgenes, separaciones entre secciones y relleno interno de componentes en toda la aplicación.

1.6 Gestión de Estado y Autenticación en el Cliente

La sesión del usuario se administra mediante la Context API nativa de React, sin recurrir a librerías externas de gestión de estado. El módulo `context/AuthContext.jsx` expone un proveedor (`AuthProvider`) que envuelve toda la aplicación en `App.jsx` y un hook de consumo (`useAuth`) utilizado por cualquier componente que necesite conocer el estado de la sesión.

Flujo de autenticación

1. Al montarse la aplicación, AuthProvider intenta recuperar una sesión previa leyendo la clave `melika_usuario` almacenada en `localStorage`.
2. El usuario envía sus credenciales desde `Login.jsx`, que invoca `api.post('/auth/login', form)` a través del cliente HTTP centralizado.
3. Si la autenticación es exitosa, el backend retorna un token JWT y los datos del usuario (incluyendo su rol); la función `login(token, datosUsuario)` del contexto persiste ambos valores en `localStorage` bajo las claves `melika_token` y `melika_usuario`, y actualiza el estado global de la aplicación.
4. Con base en el rol recibido (paciente, medico o admin), `Login.jsx` redirige automáticamente al panel correspondiente mediante `useNavigate` de React Router.
5. En cada petición posterior a la API, `apiClient.js` adjunta automáticamente el token almacenado en el encabezado `Authorization: Bearer <token>`.
6. El cierre de sesión (`logout()`) elimina ambas claves de `localStorage` y limpia el estado del contexto, sin requerir una llamada adicional al servidor, dado el carácter stateless del token JWT.

Persistencia de sesión

MELIKA no utiliza cookies de sesión ni mecanismos de servidor con estado: toda la persistencia de la sesión del cliente se resuelve mediante `localStorage`, lo que permite que el usuario permanezca autenticado entre recargas de página sin peticiones adicionales al backend.

1.7 Enrutamiento y Control de Acceso por Rol

El enrutamiento de la SPA se gestiona con React Router DOM 7 mediante `BrowserRouter` y un árbol de componentes `Route` definido en `App.jsx`. Las rutas protegidas se envuelven en tres componentes de guardia ("route guards") que verifican el rol del usuario antes de renderizar la vista solicitada:

- **RutaPaciente** — exige una sesión activa; si el usuario autenticado tiene rol médico o admin, lo redirige a su propio panel en lugar de mostrar la vista de paciente.
- **RutaMedico** — exige rol médico; cualquier otro rol es redirigido al panel de paciente.
- **RutaAdmin** — exige rol admin; cualquier otro rol es redirigido al panel de paciente.

El panel administrativo (`/admin`) utiliza rutas anidadas: `AdminLayout` actúa como contenedor persistente (con su navegación lateral) y renderiza las subrutas de gestión mediante el mecanismo de rutas hijas de React Router (`<Route index />` y rutas relativas).

Ruta	Acceso	Vista / Propósito
/	Pública	Inicio — landing page institucional.
/login	Pública	Inicio de sesión.
/registro	Pública	Registro de nuevo paciente.

Ruta	Acceso	Vista / Propósito
/verificar	Pública	Verificación de cuenta por código.
/recuperar	Pública	Solicitud de recuperación de contraseña.
/nueva-password	Pública	Definición de nueva contraseña.
/catalogo	Pública	Catálogo de medicamentos.
/activar-cuenta	Pública	Activación de cuenta por enlace.
/especialidades	Pública	Listado de especialidades médicas.
/especialidades/:id/medicos	Pública	Médicos de una especialidad específica.
/medico/:id/agenda	Paciente (RutaPaciente)	Agenda de un médico específico para reservar cita.
/dashboard	Paciente (RutaPaciente)	Panel principal del paciente.
/agendar	Paciente (RutaPaciente)	Agendamiento de una nueva cita.
/mis-citas	Paciente (RutaPaciente)	Historial de citas del paciente.
/dashboard-medico	Médico (RutaMedico)	Panel de trabajo del médico.
/admin	Administrador (RutaAdmin)	Panel administrativo — métricas generales (índice).
/admin/medicos	Administrador (RutaAdmin)	Gestión (CRUD) de médicos.
/admin/horarios	Administrador (RutaAdmin)	Gestión de horarios de atención.
/admin/usuarios	Administrador (RutaAdmin)	Gestión de usuarios registrados.
/admin/citas	Administrador (RutaAdmin)	Supervisión de todas las citas del sistema.
/admin/especialidades	Administrador (RutaAdmin)	Gestión de especialidades médicas.
/admin/medicamentos	Administrador (RutaAdmin)	Gestión del catálogo de medicamentos.

1.8 Comunicación con el Backend (Cliente API)

Toda la comunicación con el servidor pasa por un módulo centralizado, `src/lib/apiClient.js`, construido sobre la Fetch API nativa del navegador. Este cliente evita la repetición de lógica de red en cada componente y estandariza tanto el envío de peticiones como el tratamiento de errores devueltos por la API.

Aspecto	Detalle
URL base	Definida por la variable de entorno <code>VITE_API_URL</code> , con valor de respaldo <code>http://localhost:3000</code> para desarrollo local.
Formato de datos	JSON (Content-Type: <code>application/json</code>) en el cuerpo de todas las peticiones que lo requieren.
Autenticación	Token JWT enviado en el encabezado <code>Authorization: Bearer <token></code> , leído automáticamente desde <code>localStorage</code> en cada llamada.
Métodos disponibles	<code>api.get</code> , <code>api.post</code> , <code>api.put</code> , <code>api.patch</code> y <code>api.delete</code> , todos contruidos sobre una única función interna <code>request()</code> .
Manejo de respuestas vacías	Las respuestas HTTP 204 (sin contenido) se resuelven como <code>null</code> , sin intentar parsear un cuerpo JSON inexistente.
Manejo de errores	Ante una respuesta no exitosa (<code>response.ok === false</code>), se lanza un objeto <code>Error</code> enriquecido con <code>message</code> , mensaje (texto del backend), errores (arreglo de validaciones específicas, por ejemplo en historias clínicas) y <code>status</code> (código HTTP), permitiendo a cada vista reaccionar de forma diferenciada ante errores 401, 403, 422 o 500.

Adicionalmente, el módulo `services/especialidadesService.js` implementa un patrón de acceso a datos por dominio para el catálogo de especialidades y sus médicos asociados, empleando `fetch` de forma directa. Se recomienda, como oportunidad de mejora, migrar este servicio para que consuma el cliente centralizado `apiClient.js` y homogeneizar así el manejo de errores en todo el proyecto.

Carga de archivos

Para la subida de imágenes (por ejemplo, fotografías de médicos o soportes clínicos) desde los paneles administrativos, el frontend utiliza `FormData` (`multipart/form-data`) en lugar de JSON, permitiendo el envío combinado de campos de texto y archivos binarios en una misma petición.

1.9 Módulos Funcionales Principales

A continuación se describen los módulos funcionales que conforman la experiencia de usuario de MELIKA, organizados según el rol al que sirven principalmente.

Autenticación y gestión de cuenta

Comprende los flujos de registro (`Registro.jsx`), inicio de sesión (`Login.jsx`), verificación de cuenta mediante código enviado por correo electrónico (`Verificar.jsx`), activación de cuenta por enlace (`ActivarCuenta.jsx`) y recuperación de contraseña en dos pasos (`RecuperarPassword.jsx` y `NuevaPassword.jsx`). El sistema informa explícitamente al usuario

cuando su cuenta existe pero no ha sido verificada, ofreciéndole reenviar el proceso de verificación desde la propia pantalla de inicio de sesión.

Agendamiento de citas (paciente)

El flujo de agendamiento (`Agendarcita.jsx`) permite al paciente seleccionar una especialidad, un médico y una franja horaria disponible, apoyándose en el componente de calendario `FullCalendar` para representar visualmente la disponibilidad. Una vez agendada, la cita queda visible en `Mis Citas (MisCitas.jsx)`, donde el paciente también puede consultar el historial de historias clínicas asociadas a sus consultas previas.

Historias clínicas y cumplimiento normativo

El módulo de historias clínicas (`components/historias/`) constituye uno de los subsistemas más elaborados del frontend. `ModalHistoriaClinica.jsx` concentra el formulario de registro clínico, validado en tiempo real mediante `utils/validacionClinica.js`; `FormularioAclaracion.jsx` permite a un médico anexar aclaraciones a una historia ya registrada, mediante un componente montado globalmente en `App.jsx` que escucha eventos personalizados del navegador (`CustomEvent`); `PlantillaHistoriaPDF.jsx` genera un documento PDF vectorial de la historia clínica, con un diseño que declara explícitamente su alineación con la Resolución 1995 de 1999 y la Ley 2015 de 2020, normativa colombiana que regula el manejo de historias clínicas; y `VisorPDFModal.jsx` permite previsualizar dicho documento sin necesidad de descargarlo.

Panel del médico

`DashboardMedico.jsx` concentra la vista de trabajo diario del médico: consulta de su agenda, gestión de las citas que tiene asignadas y acceso directo al registro de historias clínicas de sus pacientes.

Gestión de horarios

`HorariosSemana.jsx` implementa un configurador semanal de disponibilidad, utilizado tanto por administradores como por médicos, que permite definir día por día las franjas de atención mediante el componente `ClockPicker`, con una duración de franja estandarizada de 40 minutos.

Panel administrativo

El panel de administración (`pages/admin/`) ofrece operaciones CRUD completas —creación, lectura, actualización y eliminación— sobre las entidades centrales del negocio: médicos, horarios, usuarios, citas, especialidades y medicamentos. `AdminLayout.jsx` actúa como contenedor persistente con navegación lateral, mientras que `AdminDashboard.jsx` presenta un resumen de métricas generales del sistema.

Catálogo público de medicamentos y especialidades

Las vistas `Catalogo.jsx`, `Especialidades.jsx` y `MedicosEspecialidad.jsx` conforman la exploración pública de la oferta de la institución, accesible sin necesidad de autenticarse, permitiendo a cualquier visitante conocer especialidades disponibles y el cuerpo médico asociado antes de registrarse.

1.10 Componentes de Interfaz Reutilizables

Para evitar la duplicación de código visual, MELIKA cuenta con una pequeña librería de componentes de interfaz propios en `src/components/ui/`, empleados de manera consistente en formularios y paneles a lo largo de toda la aplicación:

Componente	Descripción
Boton.jsx	Botón reutilizable con variantes visuales (primary, ghost, outline), tamaños (sm, md) y soporte para ocupar el ancho completo del contenedor (fullWidth).
DatePicker.jsx	Selector de fecha con estilo propio, utilizado en formularios de agendamiento y registro de información clínica.
TimePicker.jsx	Selector de hora con estilo propio, complementario al DatePicker.
ClockPicker.jsx	Selector de hora en formato de esfera de reloj interactiva, utilizado específicamente en la configuración semanal de horarios (HorariosSemana.jsx).
Navbar.jsx	Barra de navegación superior con menú adaptativo por rol, versión de escritorio y versión de panel deslizante (drawer) para dispositivos móviles.

Todos estos componentes se importan de forma explícita en las vistas que los requieren, sin un sistema de registro global, lo que mantiene explícitas las dependencias de cada página y facilita el análisis de impacto ante cambios.

1.11 Generación y Visualización de Documentos PDF

Uno de los requisitos más sensibles de MELIKA es la generación de historias clínicas en formato PDF, dado que estos documentos deben cumplir con la normativa colombiana vigente en materia de historia clínica. El frontend resuelve esta necesidad íntegramente en el cliente, sin depender del backend para el renderizado del documento:

- `@react-pdf/renderer` — motor de renderizado declarativo que construye el PDF a partir de componentes React (Document, Page, Text, View, StyleSheet), en lugar de convertir HTML/CSS a PDF. Esto garantiza un resultado tipográfico consistente y predecible en `PlantillaHistoriaPDF.jsx`.
- `@pdfslick/react` — visor de PDF embebido en la interfaz (`VisorPDFModal.jsx`), que permite al médico o al paciente previsualizar la historia clínica generada sin necesidad de descargarla ni abrir un lector externo.

La plantilla del documento define una paleta de color propia y una estructura de secciones clínicas estandarizada, alineada —según se documenta en los comentarios del propio código fuente— con la Resolución 1995 de 1999 y la Ley 2015 de 2020, que regulan el manejo, custodia y contenido mínimo de la historia clínica en Colombia.

Nota técnica

Dado que `@pdfslick/react` ejecuta `PDF.js` en un hilo secundario (Web Worker), el archivo `vite.config.js` configura explícitamente el formato de worker como ESM (`worker: { format: 'es' }`) y excluye la librería de la fase de pre-empaquetado de dependencias de Vite para evitar conflictos de compatibilidad.

1.12 Validación de Datos en el Cliente

El archivo `src/utills/validacionClinica.js` implementa un conjunto de reglas de validación diseñadas específicamente para el contenido clínico libre (anamnesis, antecedentes, diagnósticos), un tipo de dato que no puede validarse únicamente con reglas genéricas de longitud o formato. Este módulo se documenta a sí mismo en el código fuente como un "espejo exacto" de las validaciones equivalentes del backend (`server/src/utills/validacionesHistoria.js`), con la regla explícita de que ambos archivos deben mantenerse sincronizados manualmente ante cualquier cambio.

Reglas implementadas

- **Códigos CIE-10:** validación mediante expresión regular del formato estándar de codificación diagnóstica internacional.
- **Detección de texto trivial:** identifica campos clínicos compuestos únicamente por números, símbolos o caracteres repetidos (por ejemplo, "111" o "aaaa").
- **Detección de "ruido" alfanumérico:** reconoce patrones de escritura sin sentido clínico (por ejemplo, "asd123fgh") mediante análisis de transiciones letra-dígito y ausencia de vocales.
- **Validación de nombres propios:** reglas estrictas que solo permiten letras, espacios y guiones para campos como la firma del médico o el nombre del responsable de contacto, rechazando dígitos y correos electrónicos.
- **Términos de negación válidos:** reconoce expresiones clínicas legítimas de ausencia ("niega", "no refiere", "sin antecedentes", entre otras) para no marcarlas erróneamente como texto vacío o inválido.

Este enfoque permite que el médico reciba retroalimentación inmediata mientras completa una historia clínica, reduciendo el riesgo de que la información llegue incompleta o carente de valor semántico al backend, sin sustituir la validación de seguridad que igualmente se ejecuta del lado del servidor.

1.13 Configuración del Entorno de Build (Vite)

El archivo `client/vite.config.js` personaliza el comportamiento estándar de Vite para resolver incompatibilidades puntuales entre React 19 y determinadas dependencias del ecosistema PDF:

Configuración	Propósito
<code>plugins: [react()]</code>	Habilita el soporte de JSX y Fast Refresh mediante el plugin oficial <code>@vitejs/plugin-react</code> .
<code>resolve.alias</code>	Redirige las rutas internas de <code>use-sync-external-store</code> hacia un archivo puente propio (<code>shim-with-selector.js</code>) y hacia React directamente, evitando errores de resolución de módulos con determinadas librerías que aún dependen de ese paquete.
<code>build.outDir / commonjsOptions</code>	Define <code>dist/</code> como carpeta de salida de producción y habilita la transformación de módulos CommonJS mixtos.
<code>worker.format = 'es'</code>	Obliga a que los Web Workers (usados por PDF.js a través de <code>@pdfslick/react</code>) se compilen en formato de módulo ES, requisito de dicha librería.
<code>optimizeDeps.include / exclude</code>	Incluye explícitamente <code>@react-pdf/renderer</code> en el pre-empaquetado de esbuild y excluye <code>@pdfslick/react</code> para preservar la integridad de su Web Worker nativo.

1.14 Variables de Entorno

El frontend expone su configuración externa mediante variables de entorno con el prefijo `VITE_`, requisito de Vite para que una variable esté disponible en el código del navegador a través de `import.meta.env`:

Variable	Descripción	Valor por defecto (desarrollo)
<code>VITE_API_URL</code>	URL base del backend de MELIKA, consumida por <code>apiClient.js</code> y <code>especialidadesService.js</code> para construir cada endpoint.	<code>http://localhost:3000</code>
<code>PORT</code>	Puerto en el que Railway expone el servicio de archivos estáticos en producción (gestionado automáticamente por la plataforma, leído por el comando <code>serve -s dist -l \$PORT</code>).	

1.15 Despliegue en Railway

El frontend de MELIKA se despliega como un servicio independiente dentro de Railway (Platform as a Service), separado del servicio del backend, comunicándose con él exclusivamente a través de la URL pública definida en `VITE_API_URL` y sujeto a la política de CORS configurada en el servidor Express mediante la variable `FRONTEND_URL`.

Configuración de despliegue (client/railway.json)

Parámetro	Valor	Descripción
build.builder	NIXPACKS	Railway detecta automáticamente el proyecto Node.js/Vite y construye la imagen del contenedor sin necesidad de un Dockerfile propio.
build.buildCommand	npm run build	Ejecuta vite build, generando los archivos estáticos optimizados en la carpeta dist/.
deploy.startCommand	npm start	Ejecuta serve -s dist -l \$PORT, sirviendo la SPA como contenido estático y habilitando el modo -s (single page application) para que cualquier ruta del cliente resuelva correctamente hacia index.html.
deploy.restartPolicyType	ON_FAILURE	Railway reinicia automáticamente el servicio en caso de que el proceso finalice de forma inesperada.

Características provistas por la plataforma

- Aprovisionamiento automático del contenedor a partir del repositorio conectado en GitHub.
- Asignación de un dominio público con certificado TLS/SSL gestionado automáticamente.
- Gestión de variables de entorno (`VITE_API_URL`) desde el panel de Railway, diferenciando el valor entre entornos de desarrollo y producción.
- Integración continua: cada push a la rama principal del repositorio desencadena automáticamente un nuevo build y despliegue del frontend.
- Registro de logs de build y de ejecución accesibles desde el panel de control de Railway, útil para el diagnóstico de incidencias.

Arquitectura de despliegue en dos servicios

A diferencia de una arquitectura monolítica donde Express sirve tanto la API como los archivos estáticos, MELIKA despliega el frontend y el backend como dos servicios Railway independientes. Esta separación permite escalar, reiniciar y desplegar cada capa de forma autónoma, a costa de requerir una configuración explícita de CORS entre ambos servicios.

1.16 Calidad de Código y Convenciones

- **Linting:** el proyecto integra ESLint 10 con los plugins `eslint-plugin-react-hooks` y `eslint-plugin-react-refresh`, ejecutable mediante `npm run lint`, para detectar un uso incorrecto de Hooks y garantizar la compatibilidad con Fast Refresh durante el desarrollo.

- **Nomenclatura en español del dominio:** variables, funciones y rutas se nombran en español (usuario, rol, citas, agendar), reflejando el lenguaje del dominio de negocio y facilitando la lectura del código por parte de todo el equipo.
- **Colocalización de archivos:** cada componente o página relevante agrupa su lógica (.jsx) y sus estilos (.css) bajo el mismo nombre y carpeta, evitando la dispersión de archivos relacionados.
- **Comentarios de cabecera:** los módulos más complejos (por ejemplo, Navbar.jsx y validacionClinica.js) incluyen comentarios de cabecera que documentan reglas de negocio, historial de correcciones ("FIX v4", "FIX v5") y advertencias de sincronización con el backend, funcionando como documentación viva dentro del propio código.
- **Componentes de un único propósito:** componentes como Boton, DatePicker, TimePicker y ClockPicker se mantienen deliberadamente pequeños y enfocados en una sola responsabilidad visual, favoreciendo su reutilización entre módulos de paciente, médico y administrador.

1.17 Consideraciones y Oportunidades de Mejora

Como parte de una documentación técnica rigurosa, se identifican a continuación aspectos del frontend susceptibles de evolución en próximas iteraciones del proyecto:

- Unificar todos los accesos a datos bajo el cliente centralizado apiClient.js, migrando especialidadesService.js para heredar su manejo estandarizado de errores.
- Evaluar la incorporación de pruebas automatizadas de interfaz (por ejemplo, con Playwright o Vitest + React Testing Library), actualmente no presentes en el módulo /client.
- Formalizar la sincronización entre las reglas de validacionClinica.js y su equivalente en el backend mediante un paquete compartido o una generación automática, reduciendo el riesgo de divergencia manual señalado en los propios comentarios del código.
- Documentar mediante TypeScript o JSDoc los contratos de datos esperados por cada endpoint consumido, mejorando la detección temprana de errores de integración con el backend.



Tu salud, sin esperas ni papeleo

DOCUMENTACIÓN TÉCNICA DEL BACKEND

Arquitectura, API REST, Modelo de Datos, Seguridad y Flujos Transaccionales

Plataforma de Telemedicina y Gestión de Citas Médicas

Stack: React · Vite · Node.js · Express · PostgreSQL (pool) · Railway

Versión 1.0 — Julio de 2026

Índice

Índice	2
1. Introducción y Alcance del Backend	4
1.1 Propósito del documento	4
1.2 Alcance funcional	4
1.3 Fuera de alcance	4
2. Arquitectura General	4
2.1 Principios de diseño aplicados	5
3. Stack Tecnológico	5
4. Estructura del Proyecto	6
5. Variables de Entorno	7
6. Módulos de la API REST	7
6.1 Autenticación — /auth	7
6.2 Administración — /admin	8
6.3 Especialidades — /especialidades	9
6.4 Citas — /citas	9
6.5 Médicos — /medicos (administración) y /medico (perfil autenticado)	9
6.6 Historias Clínicas y Documentos Clínicos — /historias	10
6.7 Medicamentos — /medicamentos	11
7. Modelo de Base de Datos	11
7.1 Núcleo de identidad, catálogo y agendamiento	11
7.2 Expediente clínico	12
7.3 Historia clínica — bloques normativos (Resolución 1995/1999)	12
8. Integridad de Datos: Triggers, Auditoría y Concurrencia	13
8.1 Auditoría transparente e inmutable	13
8.2 Blindaje de concurrencia ante doble agendamiento	13
8.3 Sincronización reactiva de franjas horarias	13
8.4 Restricciones de integridad clínica (migración v5)	13
8.5 Índices de rendimiento	14
9. Mecanismos de Seguridad	14
10. Flujos Transaccionales Clave	14
10.1 Creación de una cita — blindaje ante doble agendamiento	14
10.2 Registro de una historia clínica con recetas y exámenes	15
10.3 Alta de un médico — onboarding administrado	15
11. Servicio de Correo Electrónico	15
11.1 Gmail API sobre HTTPS (canal principal)	15
11.2 SMTP vía nodemailer (canal del módulo de médicos)	16
12. Validaciones de Negocio Centralizadas	16
12.1 validacionesRegistro.js	16

12.2 validacionesHistoria.js	16
13. Despliegue en Railway	16
14. Buenas Prácticas y Decisiones Técnicas Destacables.....	17
14.1 Oportunidades de mejora identificadas.....	17

Cómo actualizar el índice: Este índice se genera a partir de los estilos de título del documento. Al abrir el archivo en Word, actualízalo con clic derecho → Actualizar campos (o F9) para reflejar la numeración de página final.

1. Introducción y Alcance del Backend

El backend de MELIKA es el servicio central que sostiene la operación de la plataforma: administra la identidad y los roles de los usuarios, coordina la agenda de citas médicas, custodia las historias clínicas digitales y expone, mediante una API REST, toda la información que consume el cliente web desarrollado en React. Su diseño prioriza tres objetivos que atraviesan cada decisión técnica documentada en las siguientes secciones: integridad clínica y legal de los registros médicos, consistencia transaccional frente a la concurrencia propia de un sistema de citas, y una separación estricta de responsabilidades entre pacientes, médicos y administradores.

A diferencia de un backend transaccional convencional (por ejemplo, un e-commerce), MELIKA opera sobre información sensible regulada: las historias clínicas se modelan siguiendo los bloques normativos de la Resolución 1995 de 1999 del Ministerio de Salud de Colombia (identificación, anamnesis, examen físico, diagnóstico codificado en CIE-10, plan de manejo y cierre legal con firma del profesional). Esta exigencia normativa se refleja directamente en el esquema de base de datos y en las validaciones de negocio descritas en las secciones 7, 8 y 10 de este documento.

1.1 Propósito del documento

Servir como referencia técnica única y actualizada del backend para cualquier desarrollador que se incorpore al proyecto, describiendo con precisión su arquitectura, sus endpoints, su modelo de datos, sus mecanismos de seguridad y las decisiones de ingeniería —incluidas sus salvaguardas ante condiciones de carrera y sus rutinas de auditoría— que sostienen su confiabilidad en producción.

1.2 Alcance funcional

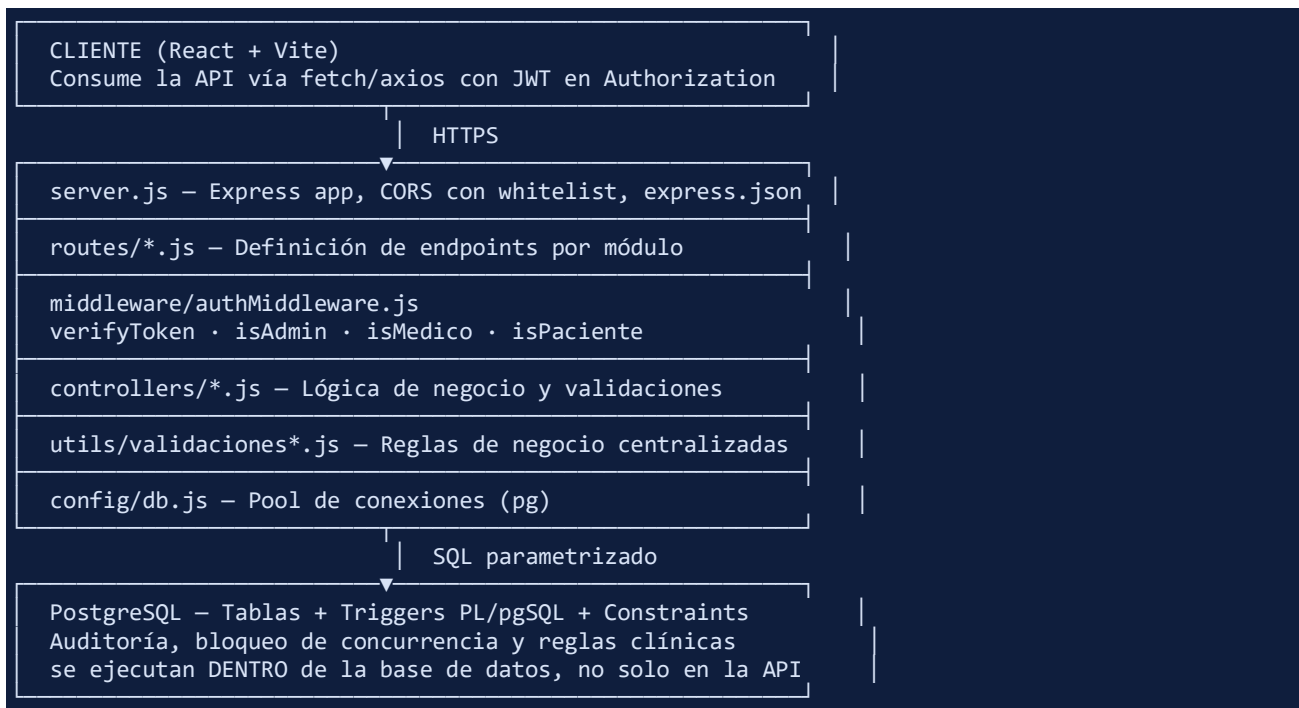
- **Gestión de identidad** — registro, verificación por código (OTP), inicio de sesión y recuperación de contraseña.
- **Agendamiento** — creación, cancelación y consulta de citas, con bloqueo automático de doble reserva a nivel de base de datos.
- registro, consulta y aclaración de historias clínicas, recetas médicas y órdenes de examen, con reglas de integridad clínica.
- catálogo de especialidades y de medicamentos (OTC y Rx), administrable desde el panel.
- **Administración** — alta controlada de médicos mediante invitación por token, gestión de horarios y métricas operativas del sistema.

1.3 Fuera de alcance

El backend no gestiona pagos ni facturación electrónica, no emite recetas con firma digital certificada ante entidad de certificación externa, y no integra directamente con historias clínicas de EPS o IPS externas; el campo `eps_aseguradora` se registra de forma informativa dentro de la historia clínica.

2. Arquitectura General

El backend implementa una arquitectura en capas sobre Node.js y Express 5, siguiendo el patrón Rutas → Middleware de autorización → Controladores → Acceso a datos, sin capa de ORM: todas las consultas se ejecutan mediante SQL parametrizado a través del driver pg y un Pool de conexiones compartido, lo que da control total sobre el rendimiento y la forma exacta de cada consulta.



La particularidad más relevante de esta arquitectura —desarrollada en detalle en la sección 8— es que MELIKA no delega toda la integridad de los datos a la capa de aplicación: reglas críticas como la prevención de doble agendamiento y la auditoría de cambios sobre citas se ejecutan mediante triggers de PostgreSQL, garantizando consistencia incluso si, en el futuro, más de un proceso o servicio escribiera directamente sobre la base de datos.

2.1 Principios de diseño aplicados

- **Separación de responsabilidades** — las rutas solo enrutan, los controladores solo orquestan, las validaciones viven en utils/ y se reutilizan entre creación y edición.
- **Orden defensivo de rutas** — toda ruta con segmento fijo (p. ej. /agenda/rango) se declara antes que su equivalente dinámica (/id) para evitar colisiones de Express; esta regla está documentada explícitamente en cada archivo de rutas.
- **Fallo explícito y acumulado** — las funciones de validación devuelven la lista completa de errores en un solo ciclo, no el primero que encuentran, reduciendo la fricción del usuario final.
- **Seguridad por capas** — bcrypt para contraseñas, JWT sin estado para sesiones, tokens de un solo uso para invitaciones y recuperación.

3. Stack Tecnológico

El backend se apoya en un conjunto reducido y deliberado de dependencias — sin ORM, sin frameworks de validación externos — priorizando control explícito sobre cada consulta SQL y cada regla de negocio. La siguiente tabla resume las tecnologías declaradas en server/package.json y su función dentro del sistema.

Tecnología	Versión	Rol en el backend
Node.js	18+ (CommonJS)	Entorno de ejecución del servidor
Express	^5.0.1	Framework HTTP: enrutamiento, middlewares y manejo de peticiones/respuestas

Tecnología	Versión	Rol en el backend
PostgreSQL	—	Motor relacional; soporta JSONB, tipos ENUM emulados vía CHECK, triggers PL/pgSQL e índices parciales/GIN
pg (node-postgres)	^8.12.0	Driver oficial; Pool de conexiones para consultas asíncronas concurrentes
jsonwebtoken	^9.0.2	Emisión y verificación de JWT con expiración de 8 horas
bcrypt	^6.0.0	Hash de contraseñas con salt rounds (factor de costo 10)
googleapis	^173.0.0	Cliente OAuth2 de Gmail API — envío de correos transaccionales vía HTTPS
nodemailer	^9.0.1	Envío de correo vía SMTP (canal usado puntualmente en el módulo de médicos)
cors	^2.8.6	Habilitación controlada de peticiones cross-origin desde el frontend
dotenv	^16.6.1	Carga de configuración sensible desde variables de entorno
nodemon	^3.1.4 (dev)	Recarga automática del servidor durante el desarrollo local

4. Estructura del Proyecto

El código del servidor vive en `server/` y sigue una organización por responsabilidad técnica (rutas, controladores, middleware, utilidades, servicios) en lugar de por entidad de negocio, lo que facilita ubicar rápidamente dónde interviene cada capa ante cualquier cambio.

Archivo / Carpeta	Descripción
<code>server.js</code>	Punto de entrada: configura CORS con whitelist, middlewares globales, registra los 7 routers y levanta el servidor HTTP en <code>0.0.0.0:PORT</code>
<code>config/db.js</code>	Instancia única del Pool de conexiones pg, configurada con variables de entorno (<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>)
<code>routes/authRoutes.js</code>	Endpoints públicos de identidad: registro, login, verificación, recuperación
<code>routes/adminRoutes.js</code>	Endpoints protegidos (<code>verifyToken + isAdmin</code>): estadísticas, usuarios, citas, horarios, especialidades, medicamentos
<code>routes/especialidadesRoutes.js</code>	Catálogo de especialidades y consulta de disponibilidad de médicos
<code>routes/citasRoutes.js</code>	Creación, consulta, cancelación y eliminación de citas del paciente autenticado
<code>routes/medicosRoutes.js</code>	Doble montaje: <code>/medicos</code> (CRUD administrado) y <code>/medico</code> (perfil, agenda y franjas del médico autenticado)
<code>routes/historiasRoutes.js</code>	Historias clínicas, notas de aclaración/evolución y documentos clínicos adjuntos
<code>routes/medicamentosRoutes.js</code>	Catálogo de medicamentos OTC/Rx y sus categorías
<code>controllers/*.js</code>	Lógica de negocio, validaciones de entrada, consultas SQL y manejo de errores por módulo (7 archivos, uno por dominio)

Archivo / Carpeta	Descripción
middleware/authMiddleware.js	verifyToken, isAdmin, isMedico e isPaciente: verificación de JWT y control de acceso basado en rol
services/emailService.js	Servicio centralizado de envío de correo mediante la Gmail API sobre HTTPS (ver sección 11)
utils/validacionesRegistro.js	Reglas de negocio para el registro de usuarios: documento, nombres, correo, fecha de nacimiento coherente
utils/validacionesHistoria.js	Motor de validación clínica: historia principal vs. notas de aclaración, rangos vitales por edad real, formato CIE-10
database/melika_db.sql	Definición completa del esquema: tablas, tipos, triggers PL/pgSQL, índices y migraciones incrementales
scripts/crearAdmin.js	Script de utilidad para provisionar la cuenta administradora inicial fuera del flujo HTTP

5. Variables de Entorno

Toda credencial y configuración sensible se gestiona exclusivamente mediante variables de entorno, cargadas con dotenv en local y definidas directamente en el panel de Railway en producción. Ninguna de ellas se versiona en el repositorio.

Variable	Descripción
PORT	Puerto HTTP en el que escucha Express (Railway lo asigna dinámicamente en producción)
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Credenciales de conexión al servicio administrado de PostgreSQL en Railway
JWT_SECRET	Clave simétrica utilizada para firmar y verificar los tokens de sesión
FRONTEND_URL	Origen autorizado en la whitelist de CORS y base para construir enlaces de activación de cuenta
EMAIL_USER / EMAIL_FROM	Cuenta de Gmail remitente de las notificaciones transaccionales
EMAIL_PASS	Contraseña de aplicación utilizada por el canal SMTP (nodemailer) del módulo de médicos
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REFRESH_TOKEN	Credenciales OAuth2 del proyecto de Google Cloud usadas por la Gmail API en services/emailService.js

6. Módulos de la API REST

La API expone 7 routers montados directamente sobre la raíz de la aplicación (sin prefijo /api). Cada tabla siguiente detalla método, ruta, propósito y nivel de protección exigido por el middleware de autenticación y autorización.

6.1 Autenticación — /auth

Método	Ruta	Descripción	Protección
POST	/auth/register	Registro de paciente con validación exhaustiva de campos y envío de código OTP por correo (expira en 15 min)	Pública
POST	/auth/login	Autenticación por correo/contraseña; verifica cuenta activa y verificada; retorna JWT (8 h)	Pública
GET	/auth/verify-code	Verificación de cuenta mediante código OTP de 6 dígitos	Pública
POST	/auth/reenviar-codigo	Reenvío de código (registro o recuperación) invalidando el anterior	Pública
POST	/auth/recuperar-password	Inicia el flujo de recuperación; respuesta neutra exista o no la cuenta (anti-enumeración)	Pública
POST	/auth/nueva-password	Cambia la contraseña validando el código de recuperación y su vigencia	Pública

6.2 Administración — /admin

Todas las rutas de este módulo aplican `router.use(verifyToken, isAdmin)` de forma global antes de sus handlers específicos.

Método	Ruta	Descripción	Protección
GET	/admin/stats	Totales operativos, citas por estado, últimas 5 citas y médicos más solicitados	Admin
GET	/admin/usuarios	Listado completo de usuarios registrados	Admin
PATCH	/admin/usuarios/:id/estado	Activa o desactiva la cuenta de un usuario	Admin
GET	/admin/citas	Listado global de citas con filtros administrativos	Admin
PATCH	/admin/citas/:id/estado	Cambio manual de estado de una cita	Admin
GET	/admin/horarios	Consulta de franjas horarias configuradas por médico	Admin
POST	/admin/horarios/masivo	Generación en bloque de franjas para una semana completa, con descansos configurables	Admin
POST	/admin/horarios	Creación de una franja horaria individual (ruta de compatibilidad)	Admin
PUT	/admin/horarios/:id	Edición de una franja libre (sin cita reservada)	Admin
DELETE	/admin/horarios/:id	Eliminación de una franja horaria	Admin
POST	/admin/especialidades	Alta de una nueva especialidad médica	Admin
PUT	/admin/especialidades/:id	Actualización de una especialidad existente	Admin
PATCH	/admin/especialidades/:id/estado	Activa o desactiva una especialidad del catálogo	Admin
POST	/admin/medicamentos	Alta de un nuevo medicamento	Admin
PUT	/admin/medicamentos/:id	Actualización de un medicamento existente	Admin

Método	Ruta	Descripción	Protección
PATCH	/admin/medicamentos/:id/estado	Activa o desactiva un medicamento del catálogo	Admin

6.3 Especialidades — /especialidades

Método	Ruta	Descripción	Protección
GET	/especialidades	Catálogo público de especialidades activas	Pública
GET	/especialidades/admin	Catálogo completo (activas e inactivas) para el panel	Admin*
GET	/especialidades/disponibilidad	Franjas disponibles de un médico en una fecha puntual	Pública
GET	/especialidades/disponibilidad-rango	Franjas disponibles de un médico dentro de un rango de fechas (calendario)	Pública
POST	/especialidades	Creación de una especialidad	Admin*
PUT	/especialidades/:id	Actualización de una especialidad	Admin*
GET	/especialidades/:id/medicos	Médicos activos asociados a una especialidad	Pública

Recomendación de endurecimiento: Las rutas de escritura de este módulo no aplican el middleware `verifyToken/isAdmin` a nivel de router; la protección administrativa recae en el consumo exclusivo desde el panel autenticado del frontend. Se recomienda blindar estas rutas con el mismo middleware usado en `/admin` para mantener consistencia defensiva en profundidad.

6.4 Citas — /citas

Método	Ruta	Descripción	Protección
GET	/citas/mis-citas	Historial de citas del paciente autenticado, con datos del médico y la especialidad	Token
GET	/citas/calendario	Citas del paciente en formato de eventos para FullCalendar, con color por estado	Token
POST	/citas	Creación de una cita; bloqueada a nivel de base de datos ante conflicto de franja	Token
PATCH	/citas/:id	Cancelación de una cita con motivo obligatorio; libera la franja automáticamente	Token
DELETE	/citas/:id	Eliminación física de una cita, solo permitida si ya está cancelada	Token

6.5 Médicos — /medicos (administración) y /medico (perfil autenticado)

Método	Ruta	Descripción	Protección
POST	/medicos/activar	Activación de cuenta médica mediante token de invitación de un solo uso	Pública
GET	/medicos	Listado de médicos para el panel administrativo	Admin
POST	/medicos	Alta de un médico: crea usuario, perfil médico y envía invitación de activación	Admin

Método	Ruta	Descripción	Protección
PUT	/medicos/:id	Actualización de los datos de un médico	Admin
PATCH	/medicos/:id/estado	Activa o desactiva a un médico	Admin
GET	/medico/perfil	Perfil del médico autenticado	Médico
GET	/medico/agenda/rango	Agenda del médico (citas + franjas) dentro de un rango, para FullCalendar	Médico
GET	/medico/agenda	Agenda diaria del médico autenticado	Médico
PATCH	/medico/citas/:id/gestionar	Registra el resultado de una cita: completada o no_asistio, con notas médicas	Médico
GET	/medico/franjas	Listado de franjas horarias propias	Médico
POST	/medico/franjas	Creación de una franja de disponibilidad	Médico
PATCH	/medico/franjas/:id	Edición de una franja propia	Médico
DELETE	/medico/franjas/:id	Eliminación de una franja propia	Médico

6.6 Historias Clínicas y Documentos Clínicos — /historias

Método	Ruta	Descripción	Protección
GET	/historias/paciente/:id_paciente	Historial clínico completo de un paciente	Token
GET	/historias/cita/:id_cita	Historia clínica asociada a una cita puntual	Token
POST	/historias	Creación de la historia clínica principal de una cita, con recetas y exámenes en una sola transacción	Médico
GET	/historias/:id/completa	Historia clínica con sus recetas y órdenes de examen asociadas	Token
POST	/historias/:id/aclaracion	Registro de una nota de aclaración o evolución sobre una historia existente	Médico
PUT	/historias/:id	Alias de compatibilidad de la ruta anterior	Médico
PATCH	/historias/gestionar-cita/:id	Gestión centralizada del resultado clínico de una cita	Médico
GET	/historias/documentos/paciente/:id_paciente	Documentos clínicos visibles de un paciente	Token
GET	/historias/documentos/:id	Detalle de un documento clínico	Token
POST	/historias/documentos	Registro de un documento clínico (fórmula, examen o documento externo)	Token
PATCH	/historias/documentos/:id/ocultar	El paciente oculta de su vista un documento externo que él mismo cargó	Paciente

Hallazgo relevante: El módulo incluye una corrección documentada en el propio código: el frontend invocaba POST /historias/:id/aclaracion para registrar notas de evolución, pero esa ruta nunca existió — solo PUT /historias/:id — lo que producía un 404 silencioso. Ambas rutas conviven hoy apuntando al mismo controlador para no romper otros consumidores.

6.7 Medicamentos — /medicamentos

Método	Ruta	Descripción	Protección
GET	/medicamentos/admin	Catálogo completo (activos e inactivos) para el panel	Pública*
GET	/medicamentos/categorias	Categorías distintas registradas en el catálogo	Pública
GET	/medicamentos	Catálogo público de medicamentos activos	Pública
GET	/medicamentos/:id	Detalle de un medicamento	Pública
POST	/medicamentos	Alta de un medicamento	Pública*
PUT	/medicamentos/:id	Actualización de un medicamento	Pública*

Recomendación de endurecimiento: Al igual que en /especialidades, las operaciones de escritura de este router no exigen JWT a nivel de middleware; dependen de que solo el panel administrativo las invoque. Se recomienda aplicar verifyToken + isAdmin explícitamente para cerrar esta superficie.

7. Modelo de Base de Datos

El esquema se compone de trece tablas relacionadas mediante llaves foráneas, con reglas de integridad reforzadas por restricciones CHECK, índices especializados (parciales y GIN) y triggers PL/pgSQL. La tabla historias_clinicas concentra el mayor nivel de detalle del modelo por ser el registro con mayor exigencia normativa del sistema.

7.1 Núcleo de identidad, catálogo y agendamiento

Tabla	Columnas y descripción
usuarios	id (PK), nombre, primer_apellido, segundo_apellido, email (UNIQUE), telefono, password_hash, rol (CHECK: paciente/medico/admin), activo, verificado, fecha_nacimiento, genero, direccion, ciudad, tipo_documento (CC/CE/PASAPORTE), numero_documento (UNIQUE), created_at, updated_at
codigos_verificacion	id (PK), email, codigo (6 dígitos), tipo (registro/recuperacion), expira_en, created_at
tokens_invitacion	id (PK), email (UNIQUE), token (UNIQUE), rol (medico/admin), usado, expira_en (72 h), created_at
especialidades	id (PK), nombre (UNIQUE), descripcion, precio_base, imagen_url, activa, created_at, updated_at
medicamentos	id (PK), nombre_comercial, principio_activo, laboratorio, categoria, tipo (OTC/Rx), descripcion, indicaciones, posologia, contraindicaciones, presentaciones, registro_invima, imagen_url, id_especialidad (FK), activo
medicos	id (PK), id_usuario (FK→usuarios, UNIQUE), id_especialidad (FK), numero_registro (UNIQUE), tarifa, calificacion, acepta_teleconsulta, acepta_presencial, biografia, anos_experiencia, activo
franjas_horarias	id (PK), id_medico (FK CASCADE), fecha, hora_inicio, hora_fin, disponible (sincronizada por trigger), CHECK hora_inicio < hora_fin

Tabla	Columnas y descripción
citas	id (PK), id_paciente (FK), id_medico (FK), id_especialidad (FK), id_franja (FK), fecha, hora_inicio, tipo_consulta (presencial/teleconsulta), estado (pendiente/completada/cancelada/no_asistio), motivo, razon_cancelacion, tarifa, notas_medicas
logs_citas	id (PK), id_cita, accion (INSERT/UPDATE/DELETE), estado_anterior, estado_nuevo, datos_anteriores (JSONB), datos_nuevos (JSONB), usuario_db, fecha_registro – poblada automáticamente por trigger de auditoría

7.2 Expediente clínico

Tabla	Columnas y descripción
historias_clinicas	Tabla central del expediente clínico, ampliada por bloques normativos (ver detalle abajo): identificación, anamnesis, examen físico, diagnóstico, plan de manejo y cierre legal con firma
documentos_clinicos	id (PK), id_historia (FK), id_paciente (FK), id_medico (FK), tipo_documento (historia_clinica/formula_medica/orden_examen/documento_externo), origen (medico/paciente), nombre_archivo, url_pdf, oculto_paciente – modelo append-only: ningún documento médico se elimina físicamente
recetas_medicas	id (PK), id_historia (FK CASCADE), medicamento, dosis, frecuencia, duracion, via_administracion, indicaciones, fecha_emision
ordenes_examenes	id (PK), id_historia (FK CASCADE), tipo_examen, nombre_examen, justificacion_clinica, fecha_emision

7.3 Historia clínica — bloques normativos (Resolución 1995/1999)

La tabla historias_clinicas se amplió de forma incremental mediante migraciones ALTER TABLE ... ADD COLUMN IF NOT EXISTS, preservando siempre los datos existentes. Sus columnas se agrupan en seis bloques funcionales:

- **Bloque 1 — Identificación administrativa** — tipo_consulta, eps_aseguradora, contacto_responsable_nombre, contacto_responsable_telefono.
- **Bloque 2 — Anamnesis expandida** — anamnesis, antecedentes_patologicos, antecedentes_quirurgicos, antecedentes_alergicos, antecedentes_familiares, antecedentes_ginecoobstetricos, habitos.
- **Bloque 3 — Examen físico y signos vitales** — tension_arterial_sistolica/diastolica, frecuencia_cardiaca, frecuencia_respiratoria, temperatura_corporal, peso_kg, talla_cm, imc (calculado automáticamente en el controlador), exploracion_por_sistemas.
- **Bloque 4 — Diagnóstico codificado** — diagnostico_cie10 (validado por formato), descripcion_diagnostico.
- **Bloque 5 — Plan de manejo** — plan_tratamiento, medicamentos_recetados (JSONB), ordenes_medicas, recomendaciones, incapacidad_dias.
- **Bloque 6 — Cierre legal** — medico_nombre_firma, medico_cedula_firma, medico_rethus_firma — exigidos en toda historia principal y en toda nota de aclaración.

Adicionalmente, cada registro clasifica su tipo_registro (historia_principal, nota_aclaracion o nota_evolucion) y conserva id_historia_original para enlazar una aclaración con el documento que corrige, sin sobrescribir ni eliminar el original — principio de trazabilidad legal del expediente.

8. Integridad de Datos: Triggers, Auditoría y Concurrencia

La decisión de ingeniería más distintiva del backend de MELIKA es que las reglas de integridad más críticas no viven únicamente en JavaScript: se ejecutan como funciones PL/pgSQL dentro de PostgreSQL, de modo que ninguna escritura —proceda de la API, de una migración o de una consola directa— pueda dejar el sistema en un estado inconsistente.

8.1 Auditoría transparente e inmutable

El trigger `trg_auditoria_citas`, asociado a `fn_auditar_citas()`, se dispara AFTER INSERT OR UPDATE OR DELETE sobre `citas` y escribe en `logs_citas` una fotografía completa de la fila (`to_jsonb`) antes y después del cambio, junto con el estado anterior y el nuevo. Esto habilita trazabilidad forense completa —quién cambió qué y cuándo— sin que el controlador tenga que registrar nada explícitamente.

8.2 Blindaje de concurrencia ante doble agendamiento

El trigger `trg_seguridad_reserva_critica`, ejecutado BEFORE INSERT sobre `citas` mediante `fn_verificar_disponibilidad_critica()`, consulta el estado real de la franja horaria en el mismo instante de la inserción. Si la franja no existe, lanza la excepción controlada `ERR_FRANJA_INEXISTENTE` (código 45001); si ya fue reservada, lanza `ERR_FRANJA_OCUPADA` (código 45002). El controlador `crearCita` intercepta ambos códigos y responde con 400 o 409 respectivamente, traduciendo un error de base de datos en un mensaje claro para el usuario final — la garantía de exclusividad ocurre en la base de datos, no en una comprobación previa en JavaScript vulnerable a condiciones de carrera.

8.3 Sincronización reactiva de franjas horarias

El trigger `trg_sincronizacion_automatica_franja`, mediante `fn_sincronizar_franja_horaria()`, mantiene `francas_horarias.disponible` siempre coherente con el estado real de las citas: la marca como ocupada al crear una cita, la libera automáticamente si la cita se cancela o se elimina. Gracias a esto, el controlador de citas nunca actualiza manualmente la disponibilidad de una franja.

8.4 Restricciones de integridad clínica (migración v5)

Sobre `historias_clinicas`, `recetas_medicas` y `ordenes_examenes` se añadieron restricciones CHECK declaradas como NOT VALID —para no romper el despliegue con datos legados— que se aplican a todo nuevo registro desde el momento de su creación:

- **Rangos clínicos válidos** — tensión arterial (50–250 / 30–150), frecuencia cardíaca (20–250), frecuencia respiratoria (5–60), temperatura (30–43 °C), peso (1–300 kg), talla (30–250 cm), incapacidad (0–180 días).
- **Coherencia de pareja** — la tensión sistólica y diastólica deben registrarse juntas o ninguna.
- **Formato CIE-10** — expresado como letra + dos dígitos + decimal opcional, validado por expresión regular.
- **Complejidad de la historia principal** — toda `historia_principal` exige anamnesis, antecedentes, examen físico, diagnóstico y firma médica completos; las notas de aclaración quedan exentas por diseño, al ser intencionalmente parciales.
- **Cierre legal obligatorio** — nombre y número RETHUS del profesional son obligatorios en cualquier documento clínico, principal o de aclaración.

8.5 Índices de rendimiento

- **idx_franjas_medico_fecha_disponible** — índice parcial sobre `franas_horarias(id_medico, fecha, disponible)` WHERE `disponible = TRUE`, que acelera la consulta de disponibilidad —la más frecuente del sistema— sin indexar franjas ya ocupadas.
- **idx_citas_paciente_fecha / idx_citas_medico_fecha** — índices compuestos sobre `citas(id_paciente, fecha)` y `citas(id_medico, fecha)` para los listados de calendario y agenda.
- **idx_logs_citas_gin_nuevos** — índice GIN sobre `logs_citas.datos_nuevos`, que permite búsquedas eficientes dentro del contenido JSONB de auditoría.

9. Mecanismos de Seguridad

Mecanismo	Descripción
Cifrado de contraseñas	bcrypt con factor de costo 10. Las contraseñas nunca se almacenan en texto plano ni se retornan en ninguna respuesta de la API.
Autenticación JWT	Tokens firmados con <code>JWT_SECRET</code> , expiración de 8 horas. Payload mínimo: <code>id</code> , <code>nombre</code> , <code>rol</code> — sin datos sensibles adicionales.
<code>verifyToken</code>	Middleware que exige el encabezado <code>Authorization: Bearer <token></code> , valida su firma y adjunta el usuario decodificado a <code>req.usuario</code> . Responde 401 ante ausencia o invalidez.
<code>isAdmin</code> / <code>isMedico</code> / <code>isPaciente</code>	Middlewares de autorización por rol, aplicados después de <code>verifyToken</code> ; responden 403 si el rol de <code>req.usuario</code> no coincide con el exigido por la ruta.
Verificación de cuenta por OTP	Código numérico de 6 dígitos con expiración de 15 minutos; el login bloquea con 403 a toda cuenta no verificada.
Recuperación de contraseña anti-enumeración	<code>solicitarRecuperacion</code> responde el mismo mensaje neutro exista o no la cuenta asociada al correo, evitando revelar qué correos están registrados.
Onboarding controlado de médicos	Los médicos no se autorregistran: el administrador crea la cuenta y el sistema emite un token de invitación de un solo uso (72 h) generado con <code>crypto.randomBytes(32)</code> .
CORS con whitelist	Solo se aceptan orígenes explícitos (<code>localhost</code> de desarrollo y <code>FRONTEND_URL</code> de producción); cualquier otro origen es rechazado por el callback de cors.

10. Flujos Transaccionales Clave

10.1 Creación de una cita — blindaje ante doble agendamiento

- **1. Solicitud** — el cliente envía `id_medico`, `id_especialidad`, `id_franja`, `fecha`, `hora_inicio`, `tipo_consulta` y `motivo`.
- **2. Validación de médico** — el controlador confirma que el médico existe y está activo, y obtiene su tarifa vigente.
- **3. Verificación atómica en base de datos** — el trigger `BEFORE INSERT` consulta el estado real de la franja en el mismo instante de la escritura.
- **4. Resolución de conflicto** — si otra petición reservó la franja microsegundos antes, PostgreSQL lanza el error 45002 y el controlador responde 409 `CONCURRENCY_CONFLICT` con un mensaje claro para reintentar con otro horario.

- **5. Confirmación** — el trigger AFTER INSERT marca la franja como no disponible y el trigger de auditoría registra el evento en logs_citas, todo en la misma transacción implícita de PostgreSQL.

10.2 Registro de una historia clínica con recetas y exámenes

- **1. Identificación del médico** — el sistema resuelve el id_medico a partir del JWT y confirma que la cita pertenece a su agenda.
- **2. Cálculo de edad real** — se calcula la edad real del paciente desde usuarios.fecha_nacimiento para validar los signos vitales contra rangos coherentes con su edad, en vez de validar "a ciegas".
- **3. Validación profesional exhaustiva** — validarHistoriaPrincipal evalúa todos los bloques clínicos y devuelve la lista completa de inconsistencias en un único response 422, no solo la primera.
- **4. Cálculo de IMC** — el IMC se calcula automáticamente a partir de peso_kg y talla_cm antes de persistir.

```
BEGIN
  INSERT INTO historias_clinicas (...)          -- historia principal
  FOR EACH receta → INSERT INTO recetas_medicas
  FOR EACH examen → INSERT INTO ordenes_examenes
  UPDATE citas SET estado = 'completada'
COMMIT
-- ante cualquier error en el bloque:
ROLLBACK → 422 (violación de integridad clínica) o 500
```

Toda la operación se ejecuta dentro de una transacción explícita (client.query('BEGIN') ... COMMIT / ROLLBACK) usando una conexión dedicada del pool, garantizando que la historia clínica, sus recetas, sus órdenes de examen y el cierre de la cita se confirmen — o se descarten — como una única unidad atómica.

10.3 Alta de un médico — onboarding administrado

- **1. Verificación de unicidad** — el administrador envía los datos del médico; el sistema verifica en paralelo (Promise.all) que el número de registro, el correo y el documento no existan previamente.
- **2. Contraseña provisional** — se genera una contraseña temporal aleatoria (nunca comunicada) con crypto.randomBytes(6) y se almacena su hash bcrypt como marcador de posición.
- **3. Creación de cuenta inactiva** — se crea el usuario con rol medico, activo=FALSE y verificado=FALSE, y su perfil médico asociado con tarifa=0 hasta que el propio médico la configure.
- **4. Invitación** — se genera un token de un solo uso con vigencia de 72 horas y se envía por correo un enlace de activación donde el médico define su contraseña real.

11. Servicio de Correo Electrónico

MELIKA utiliza dos canales de envío de correo transaccional, documentados y justificados directamente en el código fuente.

11.1 Gmail API sobre HTTPS (canal principal)

services/emailService.js implementa el envío mediante la Gmail API en lugar de SMTP tradicional. La razón es operativa: Railway, como la mayoría de plataformas cloud, bloquea a nivel de firewall los puertos SMTP salientes (25, 465, 587), por lo que nodemailer con service: 'gmail' o smtp.gmail.com nunca lograría conectar en producción, sin importar cuántos parámetros se ajusten. La Gmail API, en cambio, viaja íntegramente por HTTPS (puerto 443) — el mismo puerto que cualquier llamada REST — por lo que no es bloqueada. El mensaje se codifica en formato RFC 2822, se convierte a base64url y se envía mediante gmail.users.messages.send() autenticado con OAuth2

(GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_REFRESH_TOKEN). Este canal es el que utiliza el módulo de autenticación para los códigos OTP de registro y recuperación.

11.2 SMTP vía nodemailer (canal del módulo de médicos)

El correo de invitación de activación para nuevos médicos (medicosController.js) se envía mediante nodemailer con service: 'gmail' sobre el puerto SMTP estándar.

Recomendación de unificación: Este segundo canal está expuesto al mismo bloqueo de puertos SMTP salientes que motivó la migración a la Gmail API en el módulo de autenticación. Se recomienda unificar el envío de la invitación de médicos sobre services/emailService.js para garantizar entregabilidad consistente en Railway y reducir a un único punto de mantenimiento la lógica de correo transaccional.

12. Validaciones de Negocio Centralizadas

MELIKA evita dispersar reglas de validación entre controladores mediante dos módulos dedicados en utils/, cada uno responsable de un dominio completo y reutilizado allí donde aplica.

12.1 validacionesRegistro.js

- **Nombres no triviales** — nombre y apellido deben ser texto real: se rechazan valores puramente numéricos, solo símbolos o un carácter repetido (regex de detección de texto trivial).
- **Documento de identidad** — solo dígitos, entre 5 y 15 caracteres.
- **Correo electrónico** — formato estándar validado por expresión regular antes de cualquier consulta a base de datos.
- **Fecha de nacimiento coherente** — edad calculada y acotada entre 0 y 120 años — este dato es la única puerta de entrada de la fecha de nacimiento que luego reutiliza todo el módulo de historias clínicas para validar rangos vitales por edad real.

12.2 validacionesHistoria.js

Aplica reglas diferenciadas según el tipo de documento: una historia_principal exige la totalidad de los bloques clínicos completos, mientras que una nota aclaracion o nota evolucion se valida como intencionalmente parcial. En ambos casos, los rangos de signos vitales se contrastan contra la edad real del paciente (obtenida mediante calcularEdadAnios) y el conjunto completo de inconsistencias se devuelve en un único response, permitiendo al médico corregir todos los campos en un solo intento en lugar de recibir errores uno a uno.

13. Despliegue en Railway

El proyecto se despliega íntegramente en Railway como dos servicios independientes dentro del mismo entorno: el cliente (React + Vite) y el servidor (Node.js + Express), cada uno con su propio railway.json y su propio ciclo de build y despliegue continuo desde el repositorio de GitHub.

- **Builder** — Nixpacks detecta automáticamente el proyecto Node.js y ejecuta npm start, que a su vez ejecuta node src/server.js.
- **Binding de red** — el servidor escucha explícitamente en 0.0.0.0 y en el puerto asignado dinámicamente por Railway a través de process.env.PORT.
- **Base de datos** — provisionada como servicio administrado de Railway; las credenciales se inyectan como variables de entorno sin exponerse jamás en el repositorio.

- **CORS dinámico** — `FRONTEND_URL` se define como variable de entorno del servicio de servidor y se incorpora dinámicamente a la whitelist de CORS, evitando codificar el dominio de producción en el código fuente.
- **Restricciones de red consideradas** — el envío de correo mediante Gmail API (sección 11.1) es, precisamente, una adaptación de arquitectura motivada por las restricciones de red de este proveedor.

14. Buenas Prácticas y Decisiones Técnicas Destacables

- **Integridad garantizada en la base de datos, no solo en la API** — las reglas más críticas del sistema (prevención de doble reserva, sincronización de disponibilidad, auditoría) no dependen de que el código de aplicación las respete: se cumplen aunque exista un error en el controlador o una escritura concurrente inesperada.
- **Auditoría inmutable** — cada registro de `logs_citas` conserva una fotografía completa (JSONB) del estado anterior y posterior de la cita, habilitando trazabilidad forense sin instrumentación adicional en el código de aplicación.
- **Orden defensivo de definición de rutas** — documentado explícitamente en comentarios dentro de cada archivo de rutas, evitando que Express capture por error una ruta estática como si fuera un parámetro dinámico.
- **Validación exhaustiva, no fail-fast** — las funciones de validación agregan todos los errores encontrados antes de responder, en lugar de detenerse en el primero — una mejor experiencia para quien diligencia formularios extensos como el de historia clínica.
- **Migraciones seguras e idempotentes** — los cambios de esquema usan `IF NOT EXISTS / ADD COLUMN IF NOT EXISTS` y restricciones `NOT VALID`, permitiendo desplegar nuevas reglas sin downtime ni riesgo de romper datos ya existentes.
- **Separación estricta de roles** — `verifyToken` se combina con `isAdmin`, `isMedico` o `isPaciente` según el recurso, reflejando en el middleware la misma separación de roles que exige el negocio.

14.1 Oportunidades de mejora identificadas

- **Endurecer rutas administrativas sin JWT** — cerrar con `verifyToken + isAdmin` los endpoints de escritura de `/especialidades` y `/medicamentos`, hoy sin protección explícita a nivel de middleware.
- **Unificar el canal de correo** — consolidar el envío de correo de invitación a médicos sobre la Gmail API, evitando el canal SMTP expuesto al bloqueo de puertos de Railway.
- **Cobertura de pruebas automatizadas** — incorporar Jest/Supertest para los flujos críticos (creación de cita concurrente, transacción de historia clínica) y así proteger contra regresiones en la lógica más sensible del sistema.
- **Runner de migraciones** — centralizar la ejecución de `database/melika_db.sql` y sus migraciones incrementales en un runner versionado, en lugar de scripts ejecutados manualmente.